

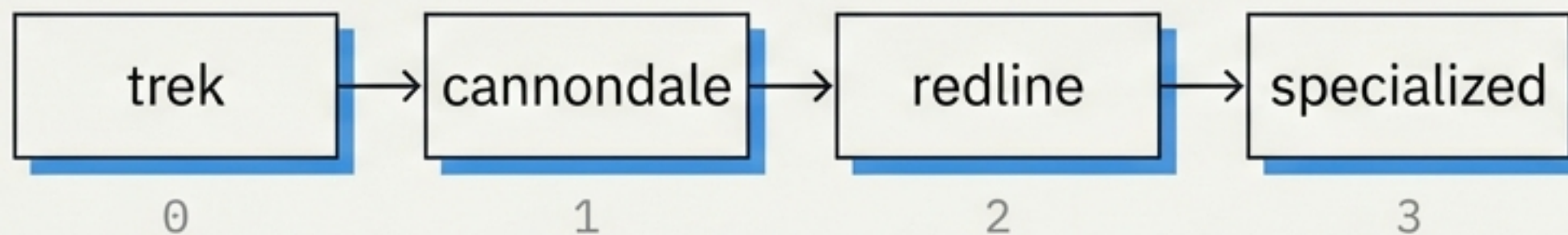
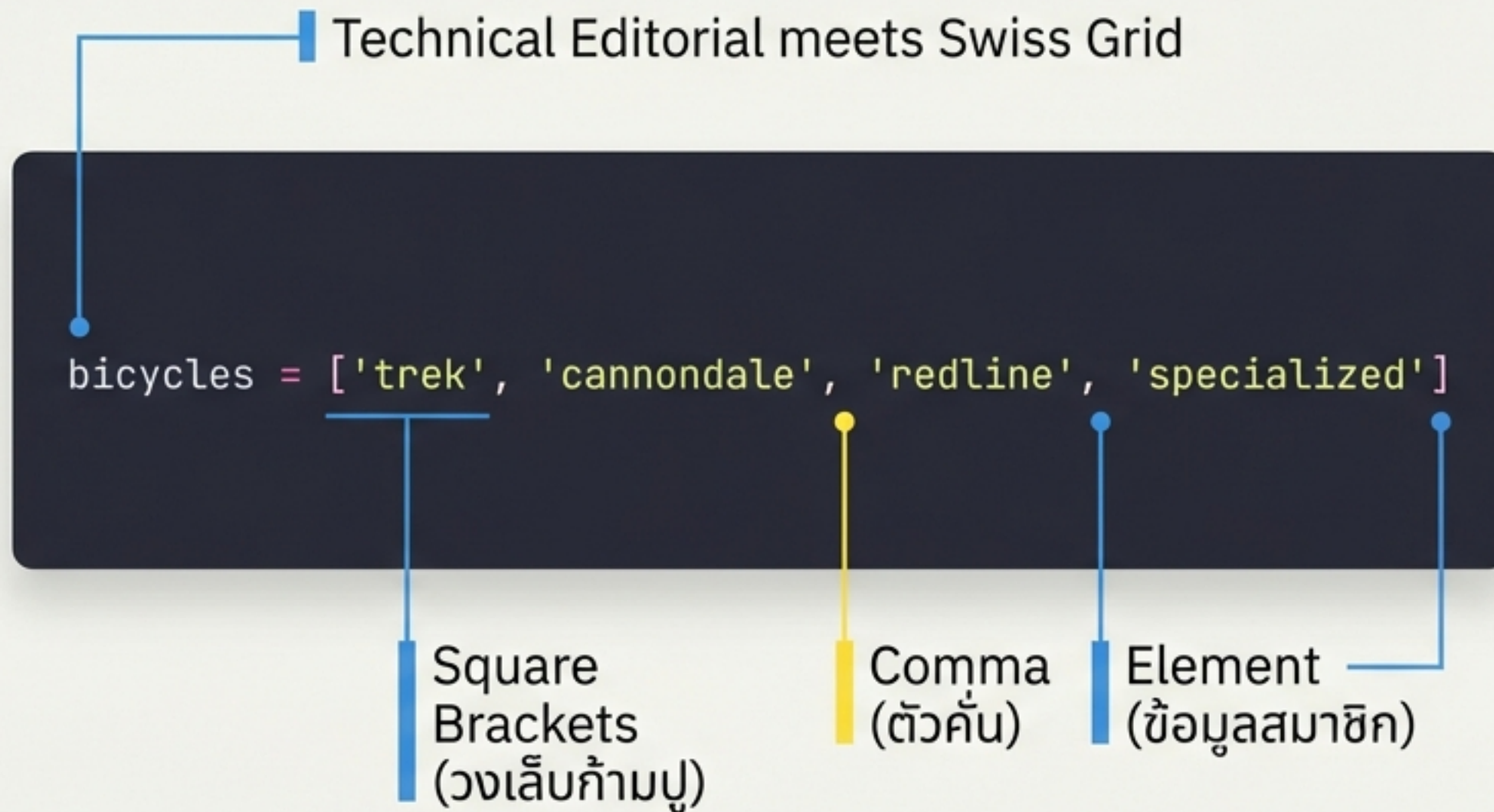
เจาะลึกโครงสร้างข้อมูล List



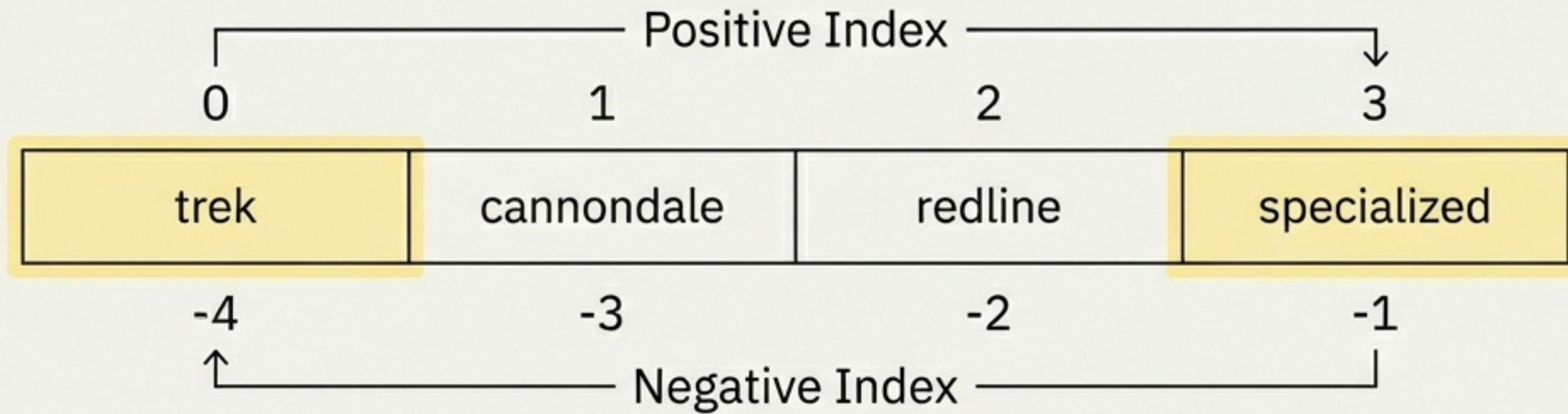
Mastering Python Lists: คู่มือฉบับสมบูรณ์สำหรับ Python Developer

ส่วนประกอบของ List (Anatomy)

List คือกลุ่มของข้อมูลที่เก็บเรียงต่อกันเป็นลำดับ สามารถเก็บข้อมูลต่างชนิดกันได้



การเข้าถึงข้อมูล (Accessing Elements)



```
print(bicycles[0])
```

-> Output: trek

```
print(bicycles[-1])
```

-> Output: specialized

```
msg = f"My first bike was a  
{bicycles[0].title()}."
```

-> Output: My first bike was a Trek.

การแก้ไขและเพิ่มข้อมูล (Modifying & Adding)

1. Modifying Elements (การแก้ไขข้อมูล)

```
motorcycles[0] = 'ducati'
```

~~['honda', 'yamaha']~~ → ['ducati', 'yamaha']

2. Appending (การเพิ่มข้อมูลท้าย List)

```
motorcycles.append('ducati')
```

'ducati' → ['honda', 'yamaha', 'suzuki'] → ['honda', 'yamaha', 'suzuki', 'ducati']

3. Inserting (การแทรกข้อมูล)

```
motorcycles.insert(0, 'ducati')
```

'ducati' ['honda', 'yamaha'] → ['ducati', 'honda', 'yamaha']

การลบข้อมูล: 3 วิธีที่แตกต่าง (Removing Elements)

Index-based Removal

1. del statement



```
del motorcycles[0]
```

ลบถาวรโดยระบุ Index

2. .pop() method



```
popped = motorcycles.pop()
```

ดึงตัวสุดท้ายออกมาใช้

Value-based Removal

3. .remove() method



```
motorcycles.remove('ducati')
```

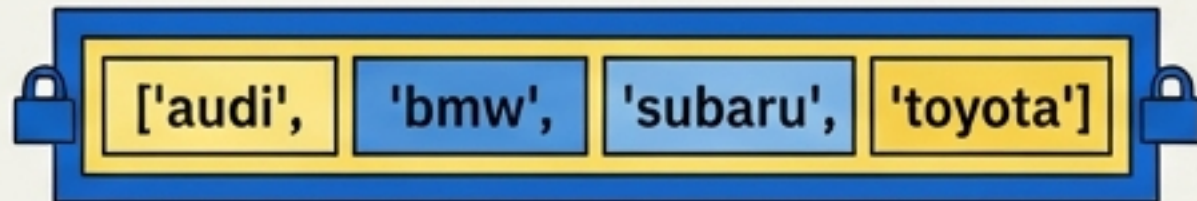
ลบโดยระบุค่า (Value)

การจัดเรียงข้อมูล (Organizing a List)



Path A (Permanent):

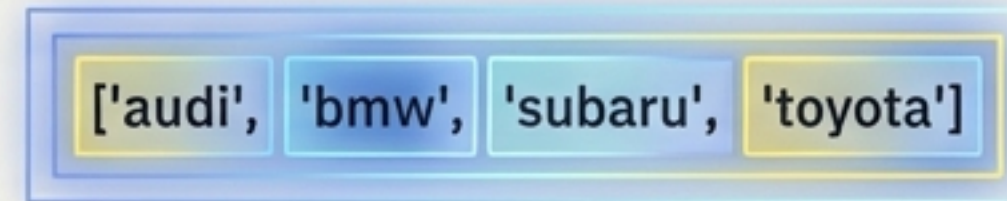
```
cars = ['bmw', 'audi', 'toyota', 'subaru']  
cars.sort()  
print(cars)
```



เปลี่ยนโครงสร้างจริง

Path B (Temporary):

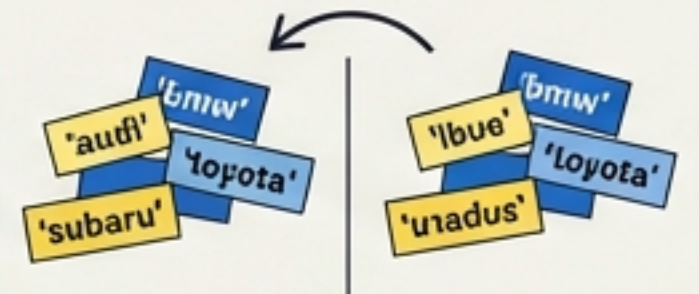
```
cars = ['bmw', 'audi', 'toyota', 'subaru']  
sorted_cars = sorted(cars)  
print(sorted_cars)
```



เรียงแค่ตอนแสดงผล

Reverse Order (การกลับลำดับ)

```
cars.reverse()
```



กลับหัวกลับหาง

การวนซ้ำด้วย Loop (Looping)

```
magicians = ['alice', 'david', 'carolina']  
for magician in magicians:  
    print(f"{magician.title()}, great trick!")
```

Don't forget
the colon!

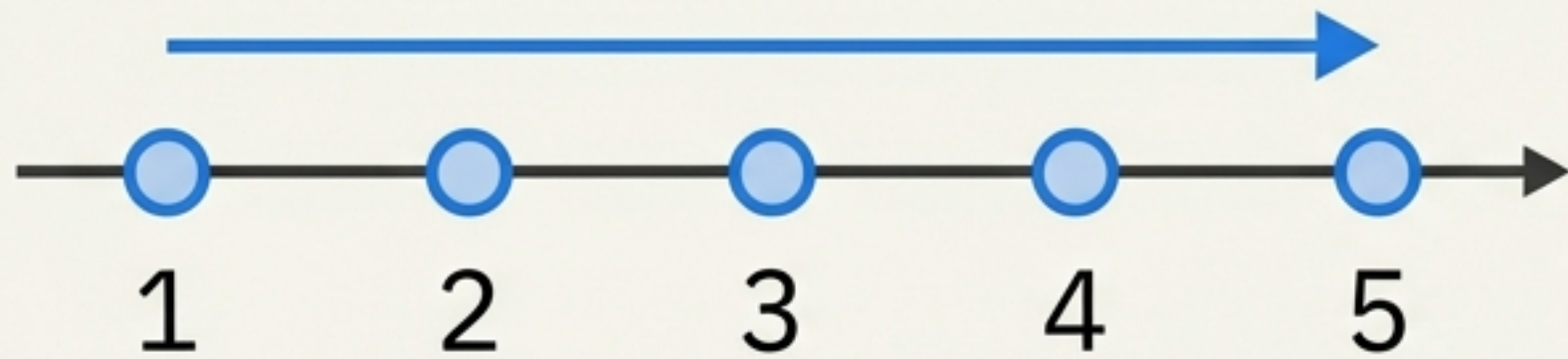
Indentation
(การย่อหน้า)
กำหนดขอบเขต



การทำงานกับตัวเลข (Numerical Lists)

int
จำนวนเต็ม
12345

Range Function



```
list(range(1, 6))
```

Stop at n-1

Simple Statistics

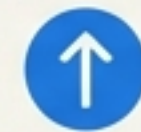
[1, 2, ... 9, 0]



min()

→ 0

จำนวนน้อยที่สุด



max()

→ 9

จำนวนมากที่สุด



sum()

→ 45

ผลรวมทั้งหมด

List Comprehensions (แบบย่อ)

Evolution

The Long Way

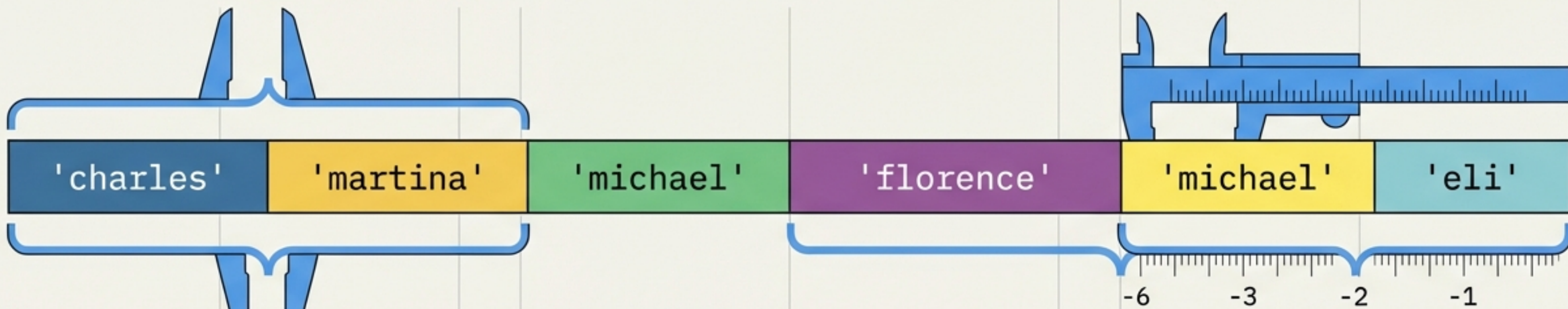
```
squares = []  
for value in range(1, 11):  
    squares.append(value**2)
```

กระชับและมีประสิทธิภาพ
(Concise & Efficient)

The
Pythonic
Way

```
squares = [value**2 for value in range(1, 11)]
```


การตัดแบ่งข้อมูล (Slicing Lists)



```
Dracula  
['charles', 'martina',  
 'michael', 'florence',  
 'eli'][0:3]
```

ดัชนีเริ่มต้น (Start index): 0 (รวม)
ดัชนีสิ้นสุด (Stop index): 3 (ไม่รวม)
ผลลัพธ์: ['charles', 'martina', 'michael']

```
Dracula  
['charles', 'martina',  
 'michael', 'florence',  
 'eli'][2:]
```

ดัชนีเริ่มต้น: 2 (รวม)
ไม่มีดัชนีสิ้นสุด: ถึงตัวสุดท้าย
ผลลัพธ์: ['michael', 'florence', 'eli']

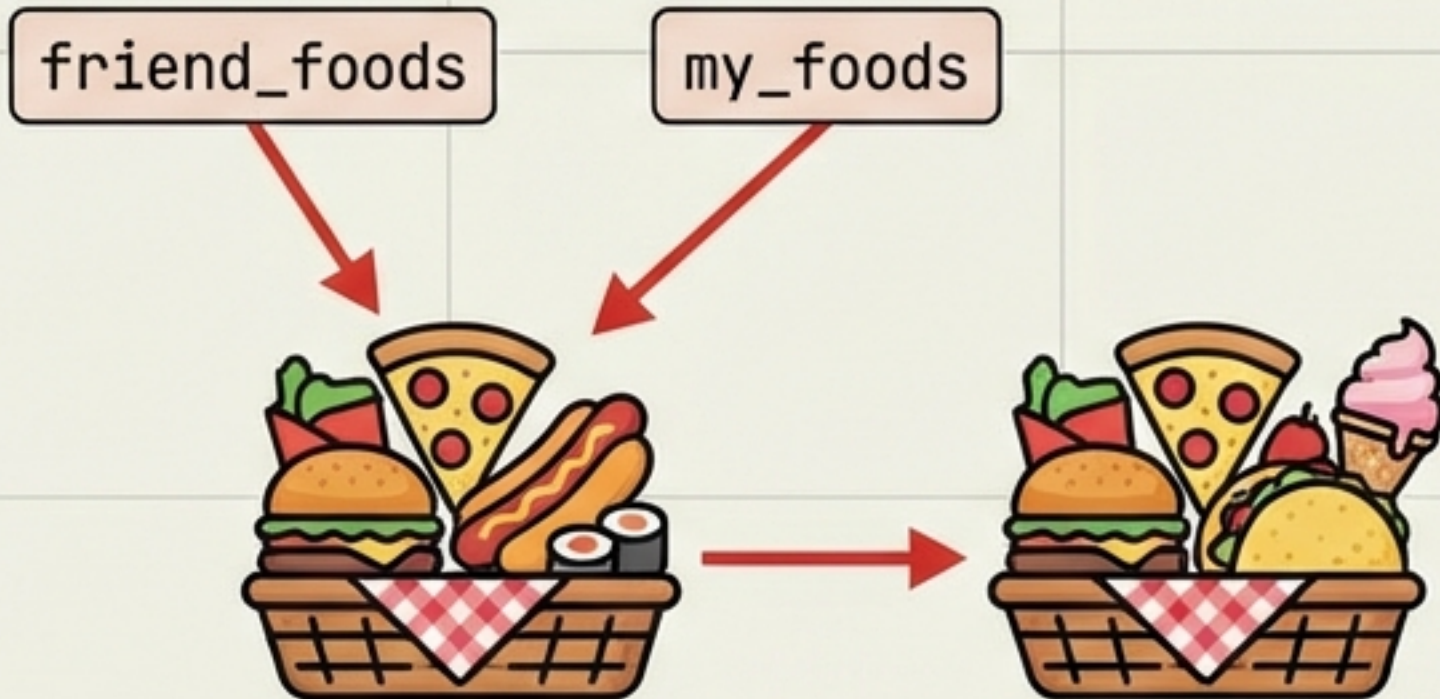
```
Dracula  
['charles', 'martina',  
 'michael', 'florence',  
 'eli'][-3:]
```

ดัชนีเริ่มต้น: -3 (รวม, นับจากท้าย)
ไม่มีดัชนีสิ้นสุด: ถึงตัวสุดท้าย
ผลลัพธ์: ['michael', 'florence', 'eli']

กับดักการคัดลอก List (Copying Lists: The Trap)

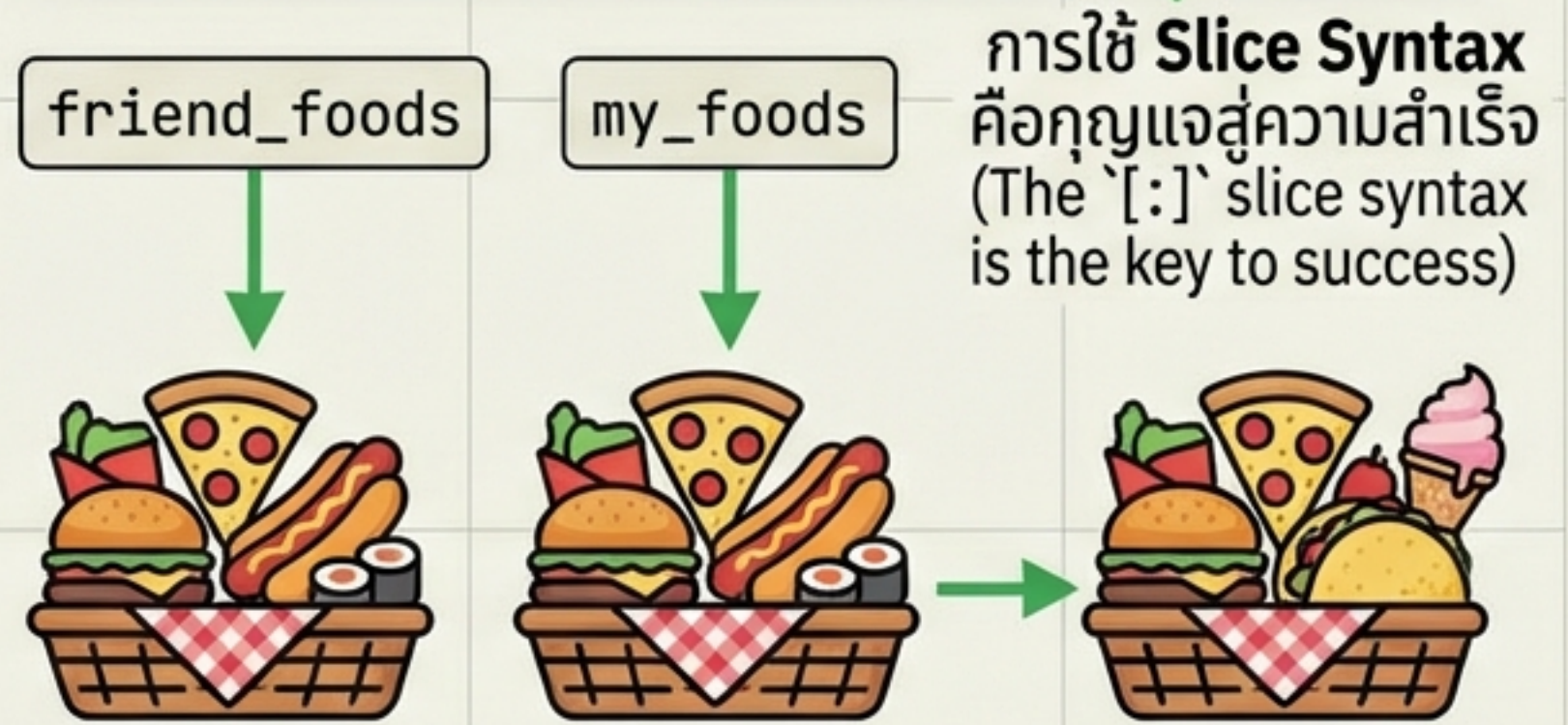
Reference (พีด)

```
friend_foods = my_foods
```



Copy (ถูก)

```
friend_foods = my_foods[:]
```



การแก้ไขไม่ส่งผลกระทบต่ออีกอัน
(Modifying one does not affect the other)

ข้อผิดพลาดที่พบบ่อย (Common Pitfalls)

Troubleshooting

IndexError

```
my_list = ['apple', 'banana', 'cherry']  
print(my_list[3])
```

list index
out of range

['apple', 'banana', 'cherry']

3

✓ Fix

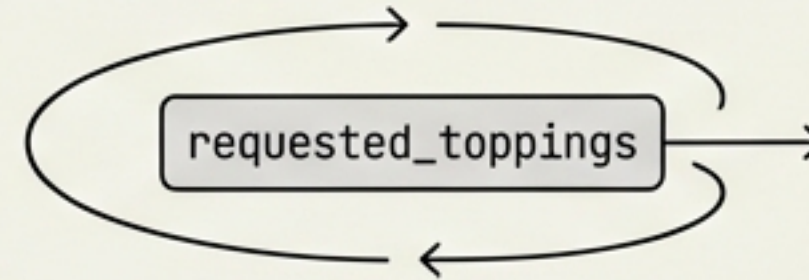
Check len() or count from 0

```
if len(my_list) > 3:  
    print(my_list[3])  
else:  
    print("Index out of range!")
```

Troubleshooting

Empty List

```
requested_toppings = []  
for topping in requested_toppings:  
    print("Adding " + topping)
```



✓ Fix

Conditional check `if requested\_toppings:`

```
requested_toppings = []  
if requested_toppings:  
    for topping in requested_toppings:  
        print("Adding " + topping)  
else:  
    print("Are you sure you want a plain pizza?")
```


Tuples vs Lists

List



```
[1, 2, 3]
```

Mutable (แก้ไขได้)



การแก้ไขส่งผล
(Modification affects it)

Tuple



```
(1, 2, 3)
```

Immutable (แก้ไขไม่ได้)



ไม่สามารถแก้ไขได้
(Cannot modify)

ใช้ Tuple เมื่อต้องการข้อมูลที่ไม่เปลี่ยนแปลง (Constants, Configs)

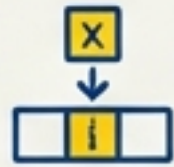
สรุปคำสั่งสำคัญ (Essential Cheat Sheet)

`.append(x)`



เพิ่ม x ต่อท้าย

`.insert(i, x)`



แทรก x
ที่ตำแหน่ง i

`del list[i]`



ลบที่ตำแหน่ง i

`.pop([i])`



ลบและคืนค่าที่ i
(หรือท้ายสุด)

`.remove(x)`



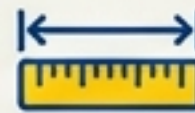
ลบค่า x
ตัวแรกที่พบ

`.sort()`



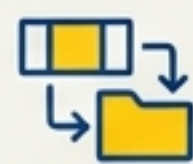
เรียงลำดับข้อมูล

`len(list)`



จำนวนสมาชิก
ทั้งหมด

`list[start:end]`



สร้าง list ใหม่
จากการตัด

พื้นฐานสู่ Data Science (Foundation for Data Science)

Python List



```
[1, "a", 3.14]
```

Flexible, Mixed Data

NumPy Arrays

```
[1 2 3]
[4 5 6]
[7 8 9]
```

```
np.array([[1 2 3],
          [4 5 6],
          [7 8 9]])
```

High Performance Calculation

Pandas DataFrame

	A	B	C
	day	temp	event
1	1/1	20	cold
2	2//19	21	hot
3	3//19	22	cool
4	4//19	22	cool

```
import pandas as pd;
data={...};
df=pd.DataFrame(data)
```

Data Manipulation

“Lists allow you to work with hundreds, thousands, and even millions of values.”